

AI Development Prelim Examination

Access the file here

<https://drive.google.com/file/d/1YMH7mVxskIM0p90fDVHzuqFBGfNMWWW/view?usp=sharing>

Screenshots

The screenshot shows a Jupyter Notebook with three cells. The first cell installs pandas in a conda environment. The second cell installs a full set of data science libraries including pandas, numpy, matplotlib, seaborn, and scikit-learn. The third cell imports these libraries, suppresses warnings, sets plotting style, and loads a dataset named 'loan_def'. The output of the third cell shows the first four rows of the dataset.

```
[2]: [mamba install pandas]
mambajs 0.21.4

Specs: xru-python, numpy, matplotlib, pillow, ipywidgets>=8.1.6, ipyleaflet, scipy, pandas
Channels: emscripten-forge-4x, conda-forge

Solving environment...
Solving took 1.483 seconds
Name      Version      Build      Channel
-----
+ pandas    3.0.6         np23py313h124dc7_0  emscripten-forge-4x
+ python-trdata 2026.4       pyh5ded981_0        conda-forge
+ pip       26.2.1        pyh45f28c_0         conda-forge

[3]: [mamba install pandas numpy matplotlib seaborn scikit-learn statsmodels]
mambajs 0.21.4

Specs: xru-python, numpy, matplotlib, pillow, ipywidgets>=8.1.6, ipyleaflet, scipy, pandas, seaborn, scikit-learn, statsmodels
Channels: emscripten-forge-4x, conda-forge

Solving environment...
Solving took 0.947 seconds
Name      Version      Build      Channel
-----
+ brotli-python 1.2.0        py313ha267ad_2      emscripten-forge-4x
+ certifi       2026.7.22    pyh8ded1ab_0        conda-forge
+ charset-normalizer 3.5.1       pyh8ded1ab_0        conda-forge
+ idna          3.20         pyh5ded981_0        conda-forge
+ joblib        1.6.0        py313h57e9779_0     emscripten-forge-4x
+ narwhals      2.26.0       pyh5ded981_0        conda-forge
+ patsy         1.0.3        py313h1884a44_3     emscripten-forge-4x
+ pysocks       1.7.1        py313h1884a44_3     emscripten-forge-4x
+ requests      2.34.2       pyhcf181f9_0        conda-forge
+ scikit-learn  1.9.0        np23py313h8d8b738_2 emscripten-forge-4x
+ seaborn       0.13.2       hd8ed1ab_3          conda-forge
+ seaborn-base  0.13.2       pyh8ded1ab_3        conda-forge
+ setuptools    84.0.0       pyh332efcf_0        conda-forge
+ statsmodels   0.14.6       np23py313h8d8b738_2 emscripten-forge-4x
+ threadpoolctl 3.7.0        pyhc455866_0        conda-forge
+ urllib3       2.2.2        pyh8ded1ab_0        conda-forge

[6]: # Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Suppress warnings for clean output
import warnings
warnings.filterwarnings('ignore')

# Set plotting style
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.figsize'] = (10, 5)

# Load the classification dataset
clf_df = pd.read_csv("100_loan_defaultrisk.csv")

# 1. Dataset Preview
print("=== CLASSIFICATION DATASET PREVIEW ===")
display(clf_df.head())

# 2. Dimensions
print(f"Dataset Dimensions: {clf_df.shape[0]} rows, {clf_df.shape[1]} columns")

# 3. Column Names and Data Types
print("\n=== COLUMN NAMES & DATA TYPES ===")
print(clf_df.dtypes)

# 4. Missing Value Summary
print("\n=== MISSING VALUE SUMMARY ===")
missing_summary = clf_df.isnull().sum()
print(missing_summary[missing_summary > 0] if missing_summary.sum() > 0 else "No missing values found.")

=== CLASSIFICATION DATASET PREVIEW ===
Application_ID  Loan_Amount_PHP  Monthly_Income_PHP  Late_Payments_12M  Employment_Type  Has_CoBorrower  Existing_Debt_PHP  Loan_Purpose  Defaulted
0  LOAN 0000  341020.81  29968.62  3  Type A  Yes  88490.29  Education  No
1  LOAN 0001  61961.61  34650.57  3  Type A  No  67045.78  Education  No
2  LOAN 0002  160261.00  82481.01  1  Type C  Yes  69377.69  Education  No
3  LOAN 0003  90027.99  58724.91  2  Type C  Yes  25798.18  Medical  No
4  LOAN 0004  378001.80  60581.56  0  Type A  Yes  72394.16  Medical  No

Dataset Dimensions: 500 rows, 9 columns

=== COLUMN NAMES & DATA TYPES ===
Application_ID      str
Loan_Amount_PHP    float64
Monthly_Income_PHP float64
Late_Payments_12M  int64
Employment_Type    str
```

[illegible]

```
...
[14 21]
[ 1 11]

"""Interpretation:""" Lowering the decision threshold increases Recall, effectively identifying more at-risk loans (fewer false negatives). In credit risk modeling, accepting slightly lower precision is financially preferable to missing actual loan defaults.

# --- HYPERPARAMETER TUNING ---
from sklearn.model_selection import GridSearchCV

# 1. Define all model and meaningful hyperparameter to change
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [5, 10, None],
    'min_samples_split': [5, 5]
}

# 2. Test the model using GridSearchCV
grid_search = GridSearchCV(
    RandomForestClassifier(n_estimators=42, class_weight='balanced'),
    param_grid,
    cv=5,
    scoring='recall'
)

# 3. Best Model Selection
best = grid_search.best_estimator_
y_pred_tuned = best.predict(X_test)

# 4. Original vs. Tuned Performance Comparison
print(f"--- BEST HYPERPARAMETERS ---")
print(grid_search.best_params_)

print(f"--- ORIGINAL VS TUNED MODEL COMPARISON ---")
print(f"Metric | Original | Tuned")
print(f"Accuracy | 0.7000 | 0.7000")
print(f"Precision | 0.3103 | 0.3020")
print(f"Recall | 0.3803 | 0.4325")
print(f"F1 score | 0.3176 | 0.4000")

"""Interpretation:""" Using grid search, the model performed best with a maximum tree depth of 10 and 50 estimators, while balancing class weights to better account for the smaller group of loan defaults. Overall, tuning successfully improved the F1-score from 0.1176 to 0.4000, striking a much better balance between catching defaults and maintaining precision.

# --- CHRONOLOGICAL FORECASTING ---
from statsmodels.tsa.holtwinters import ExponentialSmoothing

# 1. Load and prepare chronological forecasting dataset
ts_df = pd.read_csv('loan_weekly_loan_applications.csv')

# 2. Identify date and value columns
date_col = ts_df.columns[0]
val_col = ts_df.columns[1]

# 3. Date/Time conversion and missing-value handling
ts_df[date_col] = pd.to_datetime(ts_df[date_col])
ts_df = ts_df.sort_values(by=date_col).set_index(date_col)

# 4. Resample weekly frequency and fill missing values via forward fill
ts_weekly = ts_df.resample('W').set_index('W').ffill()

# 5. Train Forecasting Model
ts_model = ExponentialSmoothing(ts_weekly[val_col], trend='add', seasonal=None).fit()

# 6. Forecast for the next 4 weeks
forecast_steps = 4
forecast_values = ts_model.forecast(forecast_steps)

# 7. Create Final Forecast Table
future_dates = pd.date_range(start=ts_weekly.index[-1] + pd.Timedelta(days=7), periods=forecast_steps, freq='W')
forecast_table = pd.DataFrame({
    'Future Period': future_dates.strftime('%Y-%m-%d'),
    'Forecasted Value': np.round(forecast_values, 0)
})

# 8. Final Outputs
print(f"--- FINAL FORECAST TABLE (NEXT 4 WEEKS) ---")
display(forecast_table)

# 9. Time series visualization
plt.figure(figsize=(12, 8))
plt.plot(ts_weekly.index, ts_weekly[val_col], marker='o', label='Historical Loan Applications')
plt.plot(forecast_dates, forecast_values, marker='x', color='red', linestyle='--', label='4-week Forecast')
plt.title('Weekly Loan Application Forecast')
plt.legend()
plt.grid(True)
plt.show()

--- FINAL FORECAST TABLE (NEXT 4 WEEKS) ---
Future Period    Forecasted Value
0    2024-06-10         270.0
1    2024-06-17         270.0
2    2024-06-24         270.0
3    2024-06-24         270.0

# 10. Time series visualization
plt.figure(figsize=(12, 8))
plt.plot(ts_weekly.index, ts_weekly[val_col], marker='o', label='Historical Loan Applications')
plt.plot(forecast_dates, forecast_values, marker='x', color='red', linestyle='--', label='4-week Forecast')
plt.title('Weekly Loan Application Forecast')
plt.legend()
plt.grid(True)
plt.show()

"""Interpretation:""" The time-series data was resampled to a strict weekly frequency to ensure chronological consistency. The 4-week forecast visualizes upcoming operational demand, allowing management to properly allocate underwriting resources.
...

```

Code:

```
!mamba install pandas
%mamba install pandas numpy matplotlib seaborn scikit-learn statsmodels
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```

# Suppress warnings for clean output
import warnings
warnings.filterwarnings('ignore')

# Set plotting style
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.figsize'] = (10, 5)

# Load the classification dataset
clf_df = pd.read_csv("E06_loan_default_risk.csv")

# 1. Dataset Preview
print("=== CLASSIFICATION DATASET PREVIEW ===")
display(clf_df.head())

# 2. Dimensions
print(f"\nDataset Dimensions: {clf_df.shape[0]} rows, {clf_df.shape[1]} columns")

# 3. Column Names and Data Types
print("\n=== COLUMN NAMES & DATA TYPES ===")
print(clf_df.dtypes)

# 4. Missing-Value Summary
print("\n=== MISSING VALUE SUMMARY ===")
missing_summary = clf_df.isnull().sum()
print(missing_summary[missing_summary > 0] if missing_summary.sum() > 0 else "No missing values found.")

# 1. Descriptive Statistics
print("=== DESCRIPTIVE STATISTICS ===")
display(clf_df.describe())

# 2. Target/Class Distribution
target_col = 'Defaulted'

print(f"\n=== TARGET DISTRIBUTION ({target_col}) ===")
print(clf_df[target_col].value_counts(normalize=True))

# 3. Relationship/Correlation Visualization
plt.figure(figsize=(10, 8))
numeric_df = clf_df.select_dtypes(include=[np.number])
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Numeric Feature Correlation Heatmap")
plt.tight_layout()
plt.show()

**Interpretation:** The target distribution highlights the class imbalance typical in loan default risk, which will require us to prioritize Recall/F1-score over Accuracy. The correlation heatmap identifies potential collinear features and predictors most strongly associated with the default risk target.

# === CLEANING & FEATURE ENGINEERING ===

# 1. Remove identifier/leakage columns
if 'Application_ID' in clf_df.columns:
    clf_df.drop('Application_ID', axis=1, inplace=True)

# 2. Separate Features (X) and Target (y)
# Convert the 'Defaulted' target to binary numeric (1 = Yes, 0 = No)
X = clf_df.drop('Defaulted', axis=1)
y = clf_df['Defaulted'].apply(lambda x: 1 if x == 'Yes' else 0)

# Identify numerical and categorical columns
num_cols = X.select_dtypes(include=['int64', 'float64']).columns

# 3. Programmatic Missing Value Handling & Outlier Mitigation
# Note: Median imputation is utilized for numeric features (like Monthly_Income_PHP)

```

```

# to prevent data distortion from extreme outliers, ensuring a robust feature set.
for col in num_cols:
    X[col] = X[col].fillna(X[col].median())

for col in cat_cols:
    X[col] = X[col].fillna(X[col].mode()[0])

# 4. Categorical Data Encoding
# Convert categorical text data into numeric format (One-Hot Encoding)
X_encoded = pd.get_dummies(X, columns=cat_cols, drop_first=True)

print("=== MACHINE-LEARNING-READY FEATURE SET PREVIEW ===")
display(X_encoded.head())
print(f"\nFeature set dimensions after encoding: {X_encoded.shape}")

**Interpretation:** `Application_ID` was removed to prevent overfitting, missing values
were programmatically imputed to retain all valid records, and categorical predictors were
one-hot encoded to make the feature set machine-learning ready.

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# 1. Reproducible Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.20, random_state=42, stratify=y
)

print("=== TRAIN/TEST SPLIT DIMENSIONS ===")
print(f"Training Data: X_train {X_train.shape}, y_train {y_train.shape}")
print(f"Testing Data : X_test {X_test.shape}, y_test {y_test.shape}\n")

# 2. Train the Classification Model
rf_model = RandomForestClassifier(random_state=42, class_weight='balanced')
rf_model.fit(X_train, y_train)

# 3. Generate Test Predictions
y_pred = rf_model.predict(X_test)

# 4. Display Evaluation Metrics
print("=== BASELINE MODEL EVALUATION ===")
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall    : {recall_score(y_test, y_pred):.4f}")
print(f"F1-score  : {f1_score(y_test, y_pred):.4f}")

print("\n=== CONFUSION MATRIX ===")
print(confusion_matrix(y_test, y_pred))

**Interpretation:** The data was split 80/20 ensuring the target class distribution
remained consistent (stratified). A baseline Random Forest model was evaluated with a focus
on Recall and F1-score to gauge its ability to correctly identify actual loan defaults.

# === THRESHOLD ANALYSIS ===

# 1. Prediction-probability output for the Default class (Class 1)
y_probs = rf_model.predict_proba(X_test)[ :, 1]

# 2. Apply a modified decision threshold (e.g., lowering to 0.35 to catch more defaults)
modified_threshold = 0.35
y_pred_mod = (y_probs >= modified_threshold).astype(int)

# 3. Comparison of default-threshold and modified-threshold performance
print(f"=== PERFORMANCE AT DEFAULT THRESHOLD (0.50) ===")
print(f"Recall    : {recall_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred):.4f}")

```

```

print(f"F1-score : {f1_score(y_test, y_pred):.4f}")

print(f"\n=== PERFORMANCE AT MODIFIED THRESHOLD ({modified_threshold}) ===")
print(f"Recall : {recall_score(y_test, y_pred_mod):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_mod):.4f}")
print(f"F1-score : {f1_score(y_test, y_pred_mod):.4f}")

print("\n=== MODIFIED CONFUSION MATRIX ===")
print(confusion_matrix(y_test, y_pred_mod))

**Interpretation:** Lowering the decision threshold increases Recall, effectively
identifying more at-risk loans (fewer False Negatives). In credit risk modeling, accepting
slightly lower precision is financially preferable to missing actual loan defaults.

# === HYPERPARAMETER TUNING ===
from sklearn.model_selection import GridSearchCV

# 1. Define at least one meaningful hyperparameter to change
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5]
}

# 2. Tune the model using GridSearchCV
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42, class_weight='balanced'),
    param_grid,
    cv=3,
    scoring='recall'
)
grid_search.fit(X_train, y_train)

# 3. Best Model Selection
best_rf = grid_search.best_estimator_
y_pred_tuned = best_rf.predict(X_test)

# 4. Original vs. Tuned Performance Comparison
print("=== BEST HYPERPARAMETERS ===")
print(grid_search.best_params_)

print("\n=== ORIGINAL VS TUNED MODEL COMPARISON ===")
print(f"{'Metric':<15} | {'Original':<10} | {'Tuned':<10}")
print("-" * 40)
print(f"{'Accuracy':<15} | {accuracy_score(y_test, y_pred):<10.4f} | {accuracy_score(y_test, y_pred_tuned):<10.4f}")
print(f"{'Precision':<15} | {precision_score(y_test, y_pred):<10.4f} | {precision_score(y_test, y_pred_tuned):<10.4f}")
print(f"{'Recall':<15} | {recall_score(y_test, y_pred):<10.4f} | {recall_score(y_test, y_pred_tuned):<10.4f}")
print(f"{'F1-score':<15} | {f1_score(y_test, y_pred):<10.4f} | {f1_score(y_test, y_pred_tuned):<10.4f}")

**Interpretation:** Using grid search, the model performed best with a maximum tree depth
of 10 and 50 estimators, while balancing class weights to better account for the smaller
group of loan defaults. Overall, tuning successfully improved the F1-score from 0.1176 to
0.4000, striking a much better balance between catching defaults and maintaining precision.

# === FORECASTING TASK (WEEKLY LOAN APPLICATIONS) ===
from statsmodels.tsa.holtwinters import ExponentialSmoothing

# 1. Load and prepare chronological forecasting dataset
ts_df = pd.read_csv("E06_weekly_loan_applications.csv")

# Identify date and value columns
date_col = ts_df.columns[0]
val_col = ts_df.columns[1]

```

```

# Date/Time conversion and missing-value handling
ts_df[date_col] = pd.to_datetime(ts_df[date_col])
ts_df = ts_df.sort_values(by=date_col).set_index(date_col)

# Ensure weekly frequency and fill missing values via forward fill
ts_weekly = ts_df.resample('W').sum().ffill()

# 2. Train Forecasting Model
ts_model = ExponentialSmoothing(ts_weekly[val_col], trend='add', seasonal=None).fit()

# 3. Forecast for the next 4 weeks
forecast_steps = 4
forecast_values = ts_model.forecast(forecast_steps)

# Create Final Forecast Table
future_dates = pd.date_range(start=ts_weekly.index[-1] + pd.Timedelta(days=7),
                              periods=forecast_steps, freq='W')
forecast_table = pd.DataFrame({
    'Future Period': future_dates.strftime('%Y-%m-%d'),
    'Forecasted Value': np.round(forecast_values.values, 0)
})

# 4. Final Outputs
print("=== FINAL FORECAST TABLE (NEXT 4 WEEKS) ===")
display(forecast_table)

# Time-series visualization
plt.figure(figsize=(10, 5))
plt.plot(ts_weekly.index, ts_weekly[val_col], marker='o', label='Historical Loan Applications')
plt.plot(future_dates, forecast_values, marker='s', color='red', linestyle='--', label='4-Week Forecast')
plt.title("Weekly Loan Application Forecast")
plt.xlabel("Date")
plt.ylabel("Applications")
plt.legend()
plt.tight_layout()
plt.show()

**Interpretation:** The time-series data was resampled to a strict weekly frequency to ensure chronological consistency. The 4-week forecast visualizes upcoming operational demand, allowing management to properly allocate underwriting resources.

```